

KUBERNETES ENGINE (GKE): COMPREHENSIVE TUTORIAL

From Fundamentals to Production Deployment

TABLE OF CONTENTS

1. [Introduction to Kubernetes](#)
 2. [Google Kubernetes Engine \(GKE\) Overview](#)
 3. [Core Kubernetes Concepts](#)
 4. [GKE Architecture](#)
 5. [Lab 1: Create and Deploy to GKE Cluster](#)
 6. [Lab 2: Scaling and Load Balancing](#)
 7. [Lab 3: Storage and Configuration Management](#)
 8. [Lab 4: Networking and Services](#)
 9. [Lab 5: Deployment Strategies](#)
 10. [Lab 6: Monitoring and Logging](#)
 11. [Production Patterns](#)
 12. [Troubleshooting Guide](#)
-

INTRODUCTION TO KUBERNETES

What is Kubernetes?

Kubernetes is an open-source container orchestration platform that automates the deployment, scaling, and management of containerized applications.

The Problem Kubernetes Solves

Traditional VM Deployment:

- └ Manual provisioning
- └ Manual scaling
- └ Manual updates/rollbacks
- └ Manual failure recovery
- └ High operational overhead

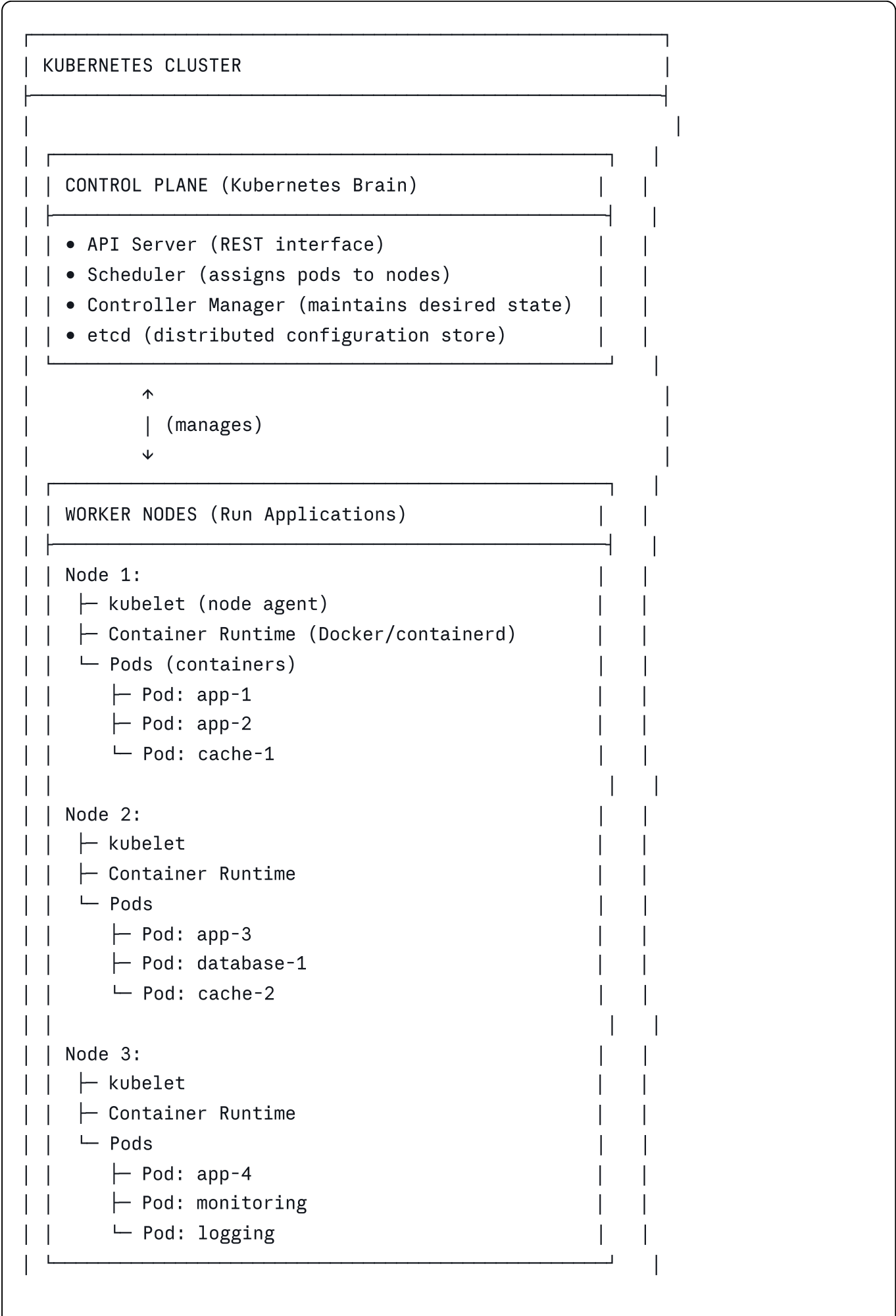
Containerized Deployment without Orchestration:

- └ Multiple containers spread across servers
- └ Manual distribution of containers
- └ No automatic scaling
- └ No health checking/restart
- └ Complex networking setup
- └ Difficult resource management

Kubernetes Solution:

- └ Automated deployment
- └ Automatic scaling (horizontal & vertical)
- └ Automated rolling updates
- └ Self-healing (restart failed containers)
- └ Automatic load balancing
- └ Resource optimization
- └ Declarative configuration

Kubernetes Architecture Overview



GOOGLE KUBERNETES ENGINE (GKE) OVERVIEW

What is GKE?

Google Kubernetes Engine (GKE) is Google Cloud's managed Kubernetes service that simplifies Kubernetes deployment, management, and scaling on Google Cloud.

GKE vs Self-Managed Kubernetes

FEATURE	GKE	Self-Managed
Control Plane Management	Google	You manage
API Server	Managed	Self-hosted
etcd Backups	Auto	Manual
Kubernetes Upgrades	One-click	Manual
Security Patches	Auto	Manual
Monitoring	Built-in	Setup required
Logging	Built-in	Setup required
Cost (Control Plane)	Free*	High (3+ nodes min)
Operational Overhead	Low	High

* GKE charges for cluster operations, node compute is separate

GKE Features

Managed Kubernetes Service

- └ Automatic scaling (Horizontal Pod Autoscaler)
- └ Automatic cluster scaling (Cluster Autoscaler)
- └ Rolling updates (zero-downtime deployments)
- └ Self-healing (automatic pod restarts)
- └ Load balancing (automatic traffic distribution)

Security & Compliance

- └ Network Policy (pod-to-pod communication control)
- └ Pod Security Policy (pod restrictions)
- └ RBAC (Role-Based Access Control)
- └ Workload Identity (pod-to-GCP service authentication)
- └ Binary Authorization (verify container images)
- └ Cloud Armor (DDoS protection)

Integration with Google Cloud

- └ Cloud Storage (object storage)
- └ Cloud SQL (managed databases)
- └ Cloud Firestore (NoSQL database)
- └ Cloud Pub/Sub (messaging)
- └ Cloud Build (CI/CD)
- └ Cloud Monitoring (observability)
- └ Cloud Logging (log aggregation)
- └ Artifact Registry (container image repository)

Development Tools

- └ kubectl (command-line tool)
- └ Cloud Code (IDE integration)
- └ Config Connector (manage GCP resources)
- └ Service Mesh (Istio integration)
- └ CI/CD integration (GitHub, GitLab, etc.)

CORE KUBERNETES CONCEPTS

1. Pods (Smallest Deployable Unit)

A **Pod** is the smallest, most basic deployable object in Kubernetes. It typically contains one container, but can contain multiple closely-coupled containers.

yaml

```
# Simple Pod definition
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
  labels:
    app: demo
spec:
  containers:
    - name: app
      image: nginx:1.14.2
      ports:
        - containerPort: 80
```

Pod Characteristics:

- Containers in a pod share network namespace (same IP address)
- Can share storage volumes
- Co-located and co-scheduled
- Usually managed by controllers (not directly)
- Ephemeral (short-lived)

2. Deployments (Stateless Applications)

A **Deployment** manages pods by ensuring desired number of replicas are running. It handles scaling, rolling updates, and rollbacks.

yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3 # Run 3 copies
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.14.2
          ports:
            - containerPort: 80
          resources:
            requests:
              cpu: 100m
              memory: 128Mi
            limits:
              cpu: 500m
              memory: 512Mi
```

Deployment Features:

- Creates and manages ReplicaSets
- Automatic rolling updates
- Rollback to previous versions
- Scaling replicas up/down
- Pause/resume deployments

3. Services (Network Access)

A **Service** provides stable network access to pods, even as they start/stop.

yaml

```
# ClusterIP Service (internal access)
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  type: ClusterIP
  selector:
    app: nginx
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80

---

# LoadBalancer Service (external access)
apiVersion: v1
kind: Service
metadata:
  name: nginx-lb
spec:
  type: LoadBalancer
  selector:
    app: nginx
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
```

Service Types:

ClusterIP (Default)

- └ Internal access only
- └ Used for pod-to-pod communication
 - └ Example: Database service

NodePort

- └ Exposes on every node's IP
- └ Fixed port on each node
 - └ Example: Testing/development

LoadBalancer

- └ Creates external load balancer
- └ Public IP address
 - └ Example: Web applications

ExternalName

- └ Maps to external DNS name
- └ No selector
 - └ Example: Third-party services

4. ConfigMaps & Secrets (Configuration)

ConfigMap stores non-sensitive configuration as key-value pairs.

yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  DATABASE_HOST: "db.example.com"
  DATABASE_PORT: "5432"
  LOG_LEVEL: "INFO"
  APP_CONFIG: |
    server:
      port: 8080
      workers: 4
```

Secret stores sensitive data (passwords, keys, tokens).

yaml

```
apiVersion: v1
kind: Secret
metadata:
  name: db-credentials
type: Opaque
stringData:
  username: admin
  password: supersecret123
```

Usage in Deployment:

```
yaml

apiVersion: apps/v1
kind: Deployment
metadata:
  name: app-deployment
spec:
  template:
    spec:
      containers:
      - name: app
        image: myapp:1.0
        # Environment variables from ConfigMap
        envFrom:
        - configMapRef:
            name: app-config
        # Environment variables from Secret
        env:
        - name: DB_PASSWORD
          valueFrom:
            secretKeyRef:
              name: db-credentials
              key: password
        # Mount ConfigMap as file
        volumeMounts:
        - name: config
          mountPath: /etc/config
      volumes:
      - name: config
        configMap:
          name: app-config
```

5. Persistent Storage

PersistentVolume (PV) and **PersistentVolumeClaim (PVC)** enable stateful applications.

```
yaml
```

```
# PersistentVolume
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-storage
spec:
  capacity:
    storage: 10Gi
  accessModes:
    - ReadWriteOnce
  gcePersistentDisk:
    pdName: gce-disk-1
    fsType: ext4

---

# PersistentVolumeClaim
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-storage
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi

---

# Pod using PVC
apiVersion: v1
kind: Pod
metadata:
  name: db-pod
spec:
  containers:
    - name: postgres
      image: postgres:13
      volumeMounts:
        - name: db-storage
          mountPath: /var/lib/postgresql/data
  volumes:
    - name: db-storage
```

```
persistentVolumeClaim:  
  claimName: pvc-storage
```

6. StatefulSets (Stateful Applications)

StatefulSet manages stateful applications with stable network identities and persistent storage.

yaml

```

apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mysql
spec:
  serviceName: mysql
  replicas: 3
  selector:
    matchLabels:
      app: mysql
  template:
    metadata:
      labels:
        app: mysql
    spec:
      containers:
        - name: mysql
          image: mysql:8.0
          ports:
            - containerPort: 3306
          env:
            - name: MYSQL_ROOT_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: mysql-secret
                  key: root-password
          volumeMounts:
            - name: data
              mountPath: /var/lib/mysql
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
        accessModes: [ "ReadWriteOnce" ]
        resources:
          requests:
            storage: 100Gi

```

StatefulSet Features:

- Stable pod names (mysql-0, mysql-1, mysql-2)
- Stable network identities (mysql-0.mysql-headless, etc.)
- Ordered creation and deletion
- Persistent storage per pod

- Suitable for: Databases, message queues, key-value stores

7. Namespaces (Resource Isolation)

Namespace provides logical isolation within a cluster.

```
yaml

# Create namespace
apiVersion: v1
kind: Namespace
metadata:
  name: production

---

# Create namespace
apiVersion: v1
kind: Namespace
metadata:
  name: development

---

# Deploy to specific namespace
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app
  namespace: production # Specify namespace
spec:
  # ... deployment spec
```

Namespace Usage:

```
bash
```

```
# List namespaces
kubectl get namespaces

# Create namespace
kubectl create namespace staging

# Deploy to namespace
kubectl apply -f deployment.yaml -n production

# Switch default namespace
kubectl config set-context --current --namespace=production

# List resources in namespace
kubectl get pods -n production
```

8. Labels & Selectors (Organization)

Labels are key-value pairs attached to resources. **Selectors** filter resources by labels.

yaml

```

apiVersion: v1
kind: Pod
metadata:
  name: web-pod
  labels:
    app: web
    version: v1
    environment: production
    tier: frontend
spec:
  containers:
  - name: web
    image: web:v1

---

# Service selecting by labels
apiVersion: v1
kind: Service
metadata:
  name: web-service
spec:
  selector:
    app: web
    environment: production
  ports:
  - port: 80
    targetPort: 8080

```

Label Selectors:

Equality-based:

- ├ app=web (matches label app=web)
- ├ environment!=dev (not equal)
- └ tier in (frontend,backend) (set membership)

Set-based:

- ├ app=web,version=v1 (AND operator)
- ├ app=web,version!=v1 (AND with inequality)
- └ app in (web,api),tier=frontend

GKE ARCHITECTURE

GKE Cluster Components



Node 4 (n1-standard-4 + GPU):

└─ Heavy ML workloads

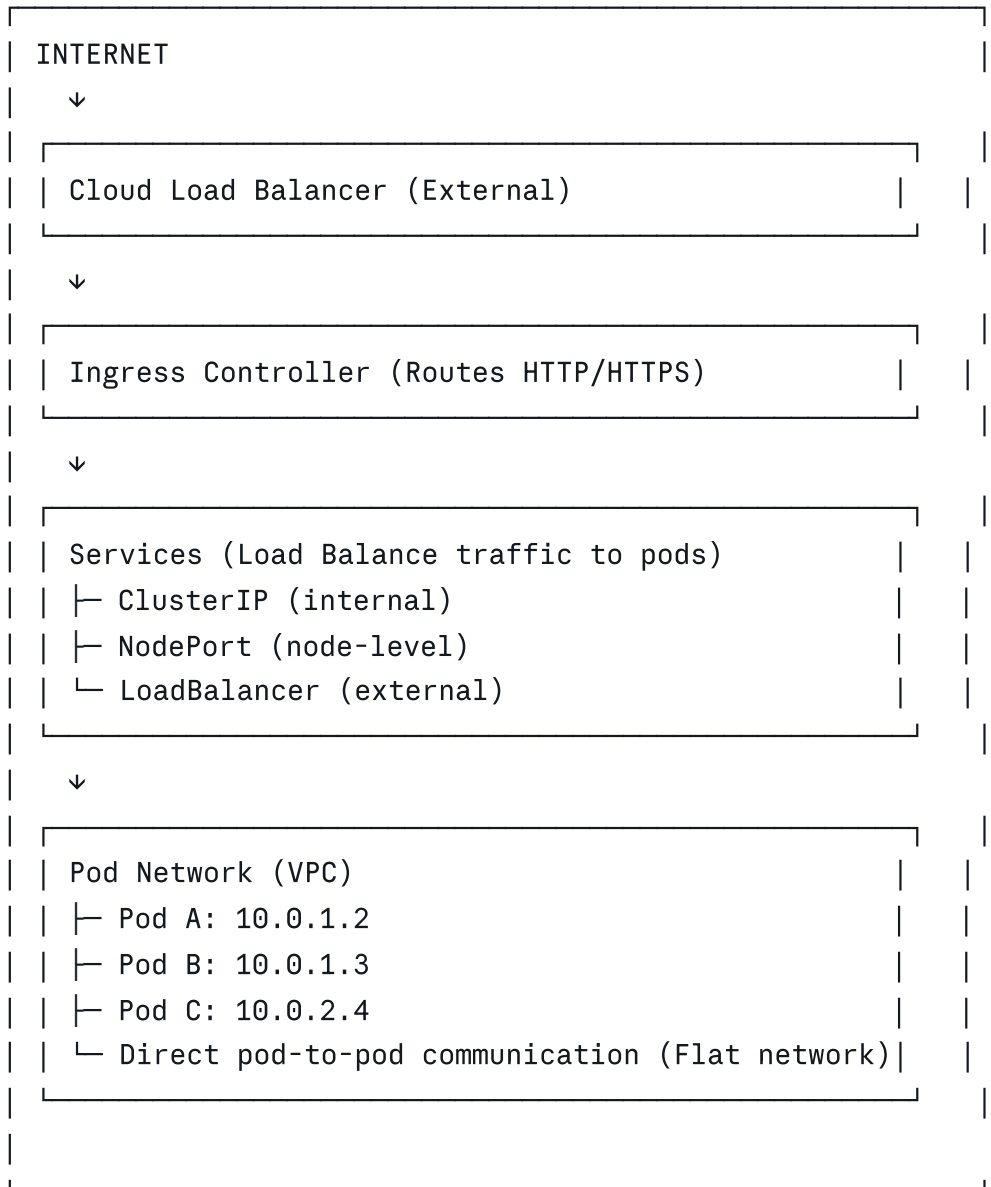
Node 5 (n1-standard-4 + GPU):

└─ Heavy ML workloads

CLUSTER ADD-ONS (Managed)

- Horizontal Pod Autoscaler (HPA)
- Cluster Autoscaler (CA)
- Monitoring & Logging (stackdriver)
- Ingress Controller
- Network Policy Controller
- DNS (CoreDNS)

GKE Networking



Network Features:

- └ VPC-native (pods get VPC IPs)
- └ Automatic pod-to-pod communication
- └ Service Discovery via DNS (service.namespace.svc.cluster.local)
- └ Network Policies (restrict traffic)
- └ Google Cloud Load Balancer integration

GKE Security Model

SECURITY LAYERS

Layer 1: Cluster Access

- └ Cloud IAM (who can access cluster)
- └ Service Accounts (cluster service accounts)
- └ RBAC (role-based access control inside cluster)

Layer 2: Network Security

- └ Network Policies (pod-to-pod communication)
- └ Firewall Rules (node-to-node communication)
- └ Cloud Armor (DDoS protection)
- └ VPC Networking (isolated network)

Layer 3: Pod Security

- └ Pod Security Policy (pod restrictions)
- └ Security Context (container runtime settings)
- └ Resource Quotas (limit resource usage)

Layer 4: Container Security

- └ Binary Authorization (verify image signatures)
- └ Vulnerability Scanning (scan container images)
- └ Container Image Signing

Layer 5: Data Security

- └ Secrets Management (encrypted secrets)
- └ Workload Identity (pod-to-GCP authentication)
- └ Encryption in transit (TLS)
- └ Encryption at rest (CMEK)

LAB 1: CREATE AND DEPLOY TO GKE CLUSTER

Lab Objective

Create a GKE cluster, deploy a simple application, and access it externally.

Prerequisites

```
bash
```

```
# Verify prerequisites
gcloud --version
kubectl version --client

# Set project ID
export PROJECT_ID=$(gcloud config get-value project)
export REGION=us-central1
export CLUSTER_NAME=my-first-cluster

echo "Project: $PROJECT_ID"
echo "Region: $REGION"
echo "Cluster: $CLUSTER_NAME"
```

Step 1: Enable Required APIs

```
bash

# Enable Kubernetes Engine API
gcloud services enable container.googleapis.com

# Enable Compute Engine API (for resources)
gcloud services enable compute.googleapis.com

# Enable Cloud Resource Manager API
gcloud services enable cloudresourcemanager.googleapis.com

# Verify APIs are enabled
gcloud services list --enabled | grep -E "container|compute|resource"
```

Expected Output:

```
compute.googleapis.com
container.googleapis.com
cloudresourcemanager.googleapis.com
```

Step 2: Create GKE Cluster

Method 1: Using gcloud CLI (Recommended for labs)

```
bash
```

```
# Create cluster with specific configuration
gcloud container clusters create $CLUSTER_NAME \
  --region $REGION \
  --num-nodes 3 \
  --machine-type n1-standard-2 \
  --enable-autorepair \
  --enable-autoupgrade \
  --addons HorizontalPodAutoscaling,HttpLoadBalancing,GcePersistentDiskCsiDri
  --workload-pool=$PROJECT_ID.svc.id.goog \
  --enable-stackdriver-kubernetes \
  --enable-ip-alias \
  --network default \
  --subnetwork default \
  --enable-autoscaling \
  --min-nodes 1 \
  --max-nodes 10 \
  --enable-autorepair \
  --labels=environment=lab,managed-by=terraform
```

Parameters Explained:

```
--region $REGION
└─ Create cluster in specific region

--num-nodes 3
└─ Initial number of nodes (can auto-scale)

--machine-type n1-standard-2
└─ VM type (2 vCPU, 7.5 GB memory)

--enable-autorepair
└─ Automatically repair unhealthy nodes

--enable-autoupgrade
└─ Automatically upgrade Kubernetes version

--addons HorizontalPodAutoscaling,HttpLoadBalancing
└─ Enable cluster add-ons

--workload-pool=$PROJECT_ID.svc.id.goog
└─ Enable Workload Identity

--enable-stackdriver-kubernetes
└─ Enable Cloud Logging and Monitoring

--enable-ip-alias
└─ Use alias IPs for pods

--enable-autoscaling --min-nodes 1 --max-nodes 10
└─ Enable cluster autoscaling

--labels=environment=lab
└─ Add labels for resource tracking
```

Cluster Creation Progress:

```
Creating cluster my-first-cluster in us-central1...
...
NAME: my-first-cluster
LOCATION: us-central1
MASTER_VERSION: 1.27.x
MASTER_STATUS: RUNNING
NODE_VERSION: 1.27.x
NUM_NODES: 3
STATUS: RUNNING
```

Creation typically takes 3-5 minutes. Watch progress:

```
bash

# Watch cluster creation
gcloud container clusters describe $CLUSTER_NAME --region $REGION --format="val

# Wait until status shows "RUNNING"
```

Step 3: Configure kubectl

After cluster creation, configure kubectl to access your cluster:

```
bash

# Get cluster credentials
gcloud container clusters get-credentials $CLUSTER_NAME \
  --region $REGION \
  --project $PROJECT_ID

# This updates ~/.kube/config with cluster access
```

Verify Configuration:

```
bash

# Test connection to cluster
kubectl cluster-info

# Expected output:
# Kubernetes control plane is running at https://...
# CoreDNS is running at https://...
```

Step 4: Verify Cluster Status

bash

Check cluster nodes

```
kubectl get nodes
```

Expected output:

# NAME	STATUS	ROLES	AGE	VERSION
# gke-my-first-cluster-default-pool-xxx	Ready	<none>	2m	v1.27.x
# gke-my-first-cluster-default-pool-yyy	Ready	<none>	2m	v1.27.x
# gke-my-first-cluster-default-pool-zzz	Ready	<none>	2m	v1.27.x

Check namespaces

```
kubectl get namespaces
```

Check cluster add-ons

```
kubectl get pods --namespace kube-system
```

Check node resources

```
kubectl describe nodes | grep -A 5 "Allocated resources"
```

Step 5: Create a Simple Deployment

Create a file named `deployment.yaml`:

yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.14.2
        ports:
        - containerPort: 80
        resources:
          requests:
            cpu: 100m
            memory: 128Mi
          limits:
            cpu: 500m
            memory: 512Mi
        livenessProbe:
          httpGet:
            path: /
            port: 80
          initialDelaySeconds: 15
          periodSeconds: 20
        readinessProbe:
          httpGet:
            path: /
            port: 80
          initialDelaySeconds: 5
          periodSeconds: 10
```

Deploy to cluster:

```
bash
```

```
# Apply deployment
kubectl apply -f deployment.yaml

# Watch deployment progress
kubectl get deployments

# Expected output:
# NAME                READY    UP-TO-DATE    AVAILABLE    AGE
# nginx-deployment    0/3      3              0            5s
# nginx-deployment    1/3      3              1            10s
# nginx-deployment    3/3      3              3            15s

# Check pods
kubectl get pods

# Expected output:
# NAME                                READY    STATUS    RESTARTS    AGE
# nginx-deployment-9456c6fb9-xxxxx    1/1      Running   0            20s
# nginx-deployment-9456c6fb9-yyyyy    1/1      Running   0            20s
# nginx-deployment-9456c6fb9-zzzzz    1/1      Running   0            20s

# Watch pod creation in real-time
kubectl get pods --watch
```

Step 6: Expose Deployment with Service

Create a file named `service.yaml`:

```
yaml

apiVersion: v1
kind: Service
metadata:
  name: nginx-service
  labels:
    app: nginx
spec:
  type: LoadBalancer
  selector:
    app: nginx
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
```

Deploy service:

```
bash

# Apply service
kubectl apply -f service.yaml

# Check service status
kubectl get services

# Expected output (initially):
# NAME                TYPE             CLUSTER-IP   EXTERNAL-IP   PORT(S)        AGE
# nginx-service       LoadBalancer     10.x.x.x     <pending>     80:xxxx/TCP    5s

# Wait for external IP to be assigned (may take 1-2 minutes)
kubectl get services --watch

# Expected output (after external IP assigned):
# NAME                TYPE             CLUSTER-IP   EXTERNAL-IP   PORT(S)        AGE
# nginx-service       LoadBalancer     10.x.x.x     34.x.x.x      80:xxxxx/TCP   2s
```

Step 7: Access Application

Once external IP is assigned, access the application:

```
bash

# Get external IP
EXTERNAL_IP=$(kubectl get service nginx-service -o jsonpath='{.status.loadBalancer.ingress[0].ip}')

echo "Access nginx at: http://$EXTERNAL_IP"

# Test with curl
curl http://$EXTERNAL_IP

# Expected output:
# <!DOCTYPE html>
# <html>
# <head>
# <title>Welcome to nginx!</title>
# ...
```

Step 8: View Pod Logs

```
bash

# Get logs from specific pod
POD_NAME=$(kubectl get pods -l app=nginx -o jsonpath='{.items[0].metadata.name}')

kubectl logs $POD_NAME

# Expected output:
# 127.0.0.1 - - [date/time] "GET / HTTP/1.1" 200 612 "-" "curl/7.x"

# Stream logs in real-time
kubectl logs -f $POD_NAME

# View logs from all pods with label
kubectl logs -l app=nginx --tail=20
```

Step 9: Execute Commands in Pod

```
bash

# Execute command in pod
kubectl exec -it $POD_NAME -- /bin/bash

# Inside container, you can run:
# ls -la
# cat /etc/nginx/nginx.conf
# ps aux

# Exit container
exit

# Or execute single command
kubectl exec $POD_NAME -- nginx -v
# Expected output: nginx version: nginx/1.14.2
```

Step 10: Monitoring Deployment

```
bash
```

```
# Check deployment details
kubectl describe deployment nginx-deployment

# Check pod details
kubectl describe pod $POD_NAME

# Check events
kubectl get events

# View resource usage
kubectl top nodes
kubectl top pods
```

Step 11: Clean Up (End of Lab)

```
bash

# Delete service (removes external load balancer)
kubectl delete service nginx-service

# Delete deployment
kubectl delete deployment nginx-deployment

# Verify deletion
kubectl get deployments
kubectl get services
kubectl get pods

# To delete entire cluster (optional, not for production)
# gcloud container clusters delete $CLUSTER_NAME --region $REGION
```

Lab 1 Summary:

- ✓ Created GKE cluster with 3 nodes
- ✓ Deployed nginx application (3 replicas)
- ✓ Exposed application with external LoadBalancer
- ✓ Accessed application from internet
- ✓ Viewed logs and executed commands
- ✓ Cleaned up resources

LAB 2: SCALING AND LOAD BALANCING

Lab Objective

Implement horizontal pod autoscaling (HPA), cluster autoscaling, and understand load balancing strategies.

Step 1: Deploy Application with Metrics Server

Metrics Server is required for HPA. Verify it's running:

```
bash

# Check if metrics-server is running
kubectl get deployment metrics-server -n kube-system

# If not present, install it (GKE usually pre-installs it)
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml

# Wait for metrics-server to be ready
kubectl wait --for=condition=available --timeout=300s deployment/metrics-server
```

Step 2: Create Deployment with Resource Requests

Create `scaling-deployment.yaml`:

```
yaml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: load-test-app
  labels:
    app: load-test
spec:
  replicas: 2
  selector:
    matchLabels:
      app: load-test
  template:
    metadata:
      labels:
        app: load-test
    spec:
      containers:
        - name: app
          image: gcr.io/google-samples/hello-app:1.0
          ports:
            - containerPort: 8080
          resources:
            requests:
              cpu: 100m      # Minimum CPU required
              memory: 128Mi  # Minimum memory required
            limits:
              cpu: 500m      # Maximum CPU allowed
              memory: 256Mi  # Maximum memory allowed
          livenessProbe:
            httpGet:
              path: /
              port: 8080
            initialDelaySeconds: 15
            periodSeconds: 20
          readinessProbe:
            httpGet:
              path: /
              port: 8080
            initialDelaySeconds: 5
            periodSeconds: 10
```

Deploy application:

```
bash
```

```
# Create deployment
kubectl apply -f scaling-deployment.yaml

# Wait for deployment to be ready
kubectl wait --for=condition=available --timeout=300s deployment/load-test-app

# Verify pods are running
kubectl get pods -l app=load-test
```

Step 3: Create Horizontal Pod Autoscaler (HPA)

Create `hpa.yaml`:

yaml

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: load-test-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: load-test-app
  minReplicas: 2      # Minimum pods
  maxReplicas: 10     # Maximum pods
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 50 # Scale up when CPU > 50%
  - type: Resource
    resource:
      name: memory
      target:
        type: Utilization
        averageUtilization: 80 # Scale up when memory > 80%
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 0
      policies:
      - type: Percent
        value: 100
        periodSeconds: 15
    scaleDown:
      stabilizationWindowSeconds: 300
      policies:
      - type: Percent
        value: 50
        periodSeconds: 60
```

Apply HPA:

```
bash
```

```
# Create HPA
kubectl apply -f hpa.yaml

# Check HPA status
kubectl get hpa

# Expected output:
# NAME                                REFERENCE                      TARGETS  MINPODS  MAXPODS  R
# load-test-hpa                      Deployment/load-test-app       5%/50%   2        10       2

# Watch HPA in real-time
kubectl get hpa --watch

# Get detailed HPA information
kubectl describe hpa load-test-hpa
```

Step 4: Create Load and Observe Scaling

First, expose the application:

```
bash

# Create service
kubectl expose deployment load-test-app \
  --type=LoadBalancer \
  --port=80 \
  --target-port=8080 \
  --name=load-test-service

# Get external IP
kubectl get service load-test-service

# Wait for external IP to be assigned
EXTERNAL_IP=$(kubectl get service load-test-service -o jsonpath='{.status.loadB
echo "Service IP: $EXTERNAL_IP"
```

Generate load from Cloud Shell:

```
bash
```

```
# Create load generation pod
kubectl run -it load-generator \
  --image=busybox:1.28 \
  --restart=Never \
  -- /bin/sh

# Inside the pod, run:
# while sleep 0.01; do wget -q -O- http://load-test-service; done

# This generates continuous load
```

Alternative: Use Apache Bench (ab) from local machine:

```
bash

# If you have ab installed locally
ab -n 10000 -c 100 http://$EXTERNAL_IP/

# This sends 10,000 requests with 100 concurrent connections
```

Step 5: Monitor Scaling Behavior

In another terminal, watch the scaling:

```
bash

# Terminal 1: Watch HPA metrics
watch kubectl get hpa

# Terminal 2: Watch pod count
watch kubectl get pods -l app=load-test

# Terminal 3: Watch events
kubectl get events --watch

# Terminal 4: Monitor resource usage
watch 'kubectl top pods -l app=load-test'
```

Expected Behavior:

Initial state (2 pods):

- └ CPU: 5%
- └ Memory: 20%
- └ Status: 2/2 replicas

Load generator starts:

- └ CPU gradually increases
- └ When CPU > 50% → Scale up triggered
- └ Every 15 seconds: Add 100% more pods (double)

Progressive scaling:

- └ 15s: 2 → 4 pods
- └ 30s: 4 → 8 pods (approaching max if still high load)
- └ 45s: 8 pods (max reached at 10)
- └ Status: Sustained at 8-10 pods while load continues

Load generator stops:

- └ CPU usage drops
- └ 300s stabilization window
- └ Scale-down begins (50% reduction per minute)
- └ 6-10 minutes: Return to minimum (2 pods)

Step 6: Check Cluster Autoscaling

Check if cluster autoscaler scales nodes:

bash

Monitor node count

`watch kubectl get nodes`

If pods can't be scheduled:

└ Cluster autoscaler detects pending pods

└ Requests new nodes from GCP

└ Adds nodes to cluster (takes 2-3 minutes)

Check cluster autoscaler logs

`kubectl logs -n kube-system -l component=cluster-autoscaler --tail=20`

Step 7: Load Balancing Strategies

Create multiple services with different load balancing types:

yaml

```
# Service 1: ClusterIP (internal only)
---
apiVersion: v1
kind: Service
metadata:
  name: load-test-internal
spec:
  type: ClusterIP
  selector:
    app: load-test
  ports:
    - port: 8080
      targetPort: 8080

# Service 2: NodePort (node-level access)
---
apiVersion: v1
kind: Service
metadata:
  name: load-test-nodeport
spec:
  type: NodePort
  selector:
    app: load-test
  ports:
    - port: 8080
      targetPort: 8080
      nodePort: 30000

# Service 3: LoadBalancer (external cloud LB)
---
apiVersion: v1
kind: Service
metadata:
  name: load-test-lb
spec:
  type: LoadBalancer
  selector:
    app: load-test
  ports:
    - port: 80
      targetPort: 8080
```

Step 8: Verify Load Distribution

```
bash

# Check which pod is handling requests
EXTERNAL_IP=$(kubectl get service load-test-lb -o jsonpath='{.status.loadBalancerIngress.ip}')

# Make multiple requests and check hostname
for i in {1..5}; do
    curl -s http://$EXTERNAL_IP | grep Hostname
done

# Expected output: Different pod names showing load distribution
# Hostname: load-test-app-7d4f5c6b8-xxxxxx
# Hostname: load-test-app-7d4f5c6b8-yyyyyy
# Hostname: load-test-app-7d4f5c6b8-zzzzzz
```

Step 9: Clean Up

```
bash

# Delete load generator pod
kubectl delete pod load-generator

# Delete services
kubectl delete service load-test-service load-test-internal load-test-nodeport

# Delete HPA
kubectl delete hpa load-test-hpa

# Delete deployment
kubectl delete deployment load-test-app
```

Lab 2 Summary:

- ✓ Deployed application with resource requests/limits
- ✓ Created Horizontal Pod Autoscaler (HPA)
- ✓ Generated load and observed auto-scaling
- ✓ Monitored scaling behavior and metrics
- ✓ Tested load balancing strategies
- ✓ Verified cluster autoscaling capability

LAB 3: STORAGE AND CONFIGURATION MANAGEMENT

Lab Objective

Learn to manage configuration with ConfigMaps and Secrets, and use persistent storage.

Step 1: Create ConfigMap

Create `configmap.yaml`:

```
yaml

apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
  labels:
    app: demo
data:
  DATABASE_HOST: "postgres.default.svc.cluster.local"
  DATABASE_PORT: "5432"
  DATABASE_NAME: "myapp_db"
  LOG_LEVEL: "INFO"
  APP_ENV: "production"
  # Multi-line configuration
  nginx.conf: |
    server {
      listen 8080;
      location / {
        proxy_pass http://backend;
      }
    }
```

Create ConfigMap:

```
bash
```

```
# Create from file
kubectl apply -f configmap.yaml

# Verify ConfigMap created
kubectl get configmap

# View ConfigMap content
kubectl describe configmap app-config

# Get specific ConfigMap value
kubectl get configmap app-config -o jsonpath='{.data.LOG_LEVEL}'
```

Step 2: Create Secret

Create `secret.yaml`:

```
yaml

apiVersion: v1
kind: Secret
metadata:
  name: db-credentials
type: Opaque
stringData:
  username: admin
  password: secretpassword123
  connection-string: "postgresql://admin:secretpassword123@postgres:5432/myapp_
```

Create Secret:

```
bash
```

```
# Create from file (note: stringData is not recommended for sensitive data in p
kubectl apply -f secret.yaml

# Verify Secret created
kubectl get secrets

# View Secret content (stored as base64)
kubectl get secret db-credentials -o yaml

# Decode secret (careful with sensitive data)
kubectl get secret db-credentials -o jsonpath='{.data.password}' | base64 -d
```

Better Practice: Create Secret with kubectl (more secure):

```
bash

# Create secret directly without storing in file
kubectl create secret generic db-credentials-secure \
  --from-literal=username=admin \
  --from-literal=password=secretpassword123 \
  --from-literal=connection-string='postgresql://admin:secretpassword123@postgr
  --dry-run=client -o yaml | kubectl apply -f -

# Verify
kubectl get secret db-credentials-secure
```

Step 3: Create Deployment Using ConfigMap and Secret

Create `app-deployment.yaml`:

```
yaml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: config-demo-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: config-demo
  template:
    metadata:
      labels:
        app: config-demo
    spec:
      containers:
        - name: app
          image: gcr.io/google-samples/hello-app:2.0
          ports:
            - containerPort: 8080

# Method 1: Environment variables from ConfigMap
env:
  - name: DATABASE_HOST
    valueFrom:
      configMapKeyRef:
        name: app-config
        key: DATABASE_HOST
  - name: DATABASE_PORT
    valueFrom:
      configMapKeyRef:
        name: app-config
        key: DATABASE_PORT
  - name: LOG_LEVEL
    valueFrom:
      configMapKeyRef:
        name: app-config
        key: LOG_LEVEL

# Environment variables from Secret
- name: DB_USERNAME
  valueFrom:
    secretKeyRef:
      name: db-credentials
      key: username
```

```
- name: DB_PASSWORD
  valueFrom:
    secretKeyRef:
      name: db-credentials
      key: password

# Method 2: Mount ConfigMap as volume (file)
volumeMounts:
- name: config-volume
  mountPath: /etc/config
- name: secret-volume
  mountPath: /etc/secrets
  readOnly: true

volumes:
- name: config-volume
  configMap:
    name: app-config
    items:
      - key: nginx.conf
        path: nginx.conf
- name: secret-volume
  secret:
    secretName: db-credentials
    defaultMode: 0400 # Read-only for owner
```

Deploy application:

```
bash

# Create deployment
kubectl apply -f app-deployment.yaml

# Wait for deployment
kubectl wait --for=condition=available --timeout=300s deployment/config-demo-ap

# Verify pods are running
kubectl get pods -l app=config-demo
```

Step 4: Verify Configuration in Pods

```
bash
```

```
# Execute into pod to verify configuration
POD_NAME=$(kubectl get pods -l app=config-demo -o jsonpath='{.items[0].metadata.name}')

# Check environment variables
kubectl exec $POD_NAME -- env | grep -E "DATABASE|DB_|LOG_LEVEL"

# Check mounted config files
kubectl exec $POD_NAME -- ls -la /etc/config
kubectl exec $POD_NAME -- cat /etc/config/nginx.conf

# Check mounted secret files
kubectl exec $POD_NAME -- ls -la /etc/secrets
kubectl exec $POD_NAME -- cat /etc/secrets/username
kubectl exec $POD_NAME -- cat /etc/secrets/password
```

Step 5: Create PersistentVolumeClaim

Create `pvc.yaml`:

```
yaml

apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: data-pvc
spec:
  accessModes:
    - ReadWriteOnce      # Only one pod can write
  resources:
    requests:
      storage: 10Gi      # 10 GB storage
  storageClassName: standard # Standard persistent disk
```

Create PVC:

```
bash
```

```
# Create PVC
kubectl apply -f pvc.yaml

# Verify PVC created
kubectl get pvc

# Expected output:
# NAME          STATUS    VOLUME          CAPACITY   ACCE
# data-pvc      Bound     pvc-abc123...    10Gi       RWO

# If Status is "Pending", PV will be created automatically when pod uses it
```

Step 6: Create Pod with Persistent Storage

Create `stateful-pod.yaml`:

```
yaml

apiVersion: v1
kind: Pod
metadata:
  name: data-pod
spec:
  containers:
    - name: data-app
      image: busybox:1.28
      command: ['sh', '-c', 'while true; do echo "Current time: $(date)" >> /data
      volumeMounts:
        - name: data-storage
          mountPath: /data
      volumes:
        - name: data-storage
          persistentVolumeClaim:
            claimName: data-pvc
```

Deploy pod:

```
bash
```

```
# Create pod
kubectl apply -f stateful-pod.yaml

# Wait for pod to be ready
kubectl wait --for=condition=Ready pod/data-pod --timeout=60s

# Check logs being written
sleep 10
kubectl exec data-pod -- tail -f /data/log.txt

# Stop after viewing logs (Ctrl+C)
```

Step 7: Verify Persistent Storage

```
bash

# List files in mounted directory
kubectl exec data-pod -- ls -lah /data

# View file contents
kubectl exec data-pod -- cat /data/log.txt

# File should contain multiple entries, showing data was preserved
```

Step 8: Delete and Recreate Pod (Data Persists)

```
bash

# Delete the pod
kubectl delete pod data-pod

# Verify pod is deleted
kubectl get pods

# Recreate pod with same volume mount
kubectl apply -f stateful-pod.yaml

# Check that data is still there
sleep 5
kubectl exec data-pod -- tail /data/log.txt

# Old entries should be present, with new entries appended
```

Step 9: Create StatefulSet with Persistent Storage

Create `statefulset.yaml`:

yaml

```
apiVersion: v1
kind: Service
metadata:
  name: mysql
spec:
  clusterIP: None # Headless service for StatefulSet
  selector:
    app: mysql
  ports:
    - port: 3306
      targetPort: 3306
```

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mysql
spec:
  serviceName: mysql
  replicas: 1 # Single instance for lab
  selector:
    matchLabels:
      app: mysql
  template:
    metadata:
      labels:
        app: mysql
    spec:
      containers:
        - name: mysql
          image: mysql:5.7
          ports:
            - containerPort: 3306
          env:
            - name: MYSQL_ROOT_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: mysql-secret
                  key: root-password
          volumeMounts:
            - name: data
              mountPath: /var/lib/mysql
      volumeClaimTemplates:
```

```
- metadata:
  name: data
spec:
  accessModes: ["ReadWriteOnce"]
  resources:
    requests:
      storage: 20Gi
```

Deploy StatefulSet:

bash

```
# Create MySQL secret first
kubectl create secret generic mysql-secret \
  --from-literal=root-password=rootpassword123 \
  --dry-run=client -o yaml | kubectl apply -f -

# Create StatefulSet
kubectl apply -f statefulset.yaml

# Wait for StatefulSet
kubectl wait --for=condition=Ready pod/mysql-0 --timeout=300s

# Verify StatefulSet created persistent storage
kubectl get pvc

# Expected output:
# NAME              STATUS    VOLUME             CAPACITY   ACCESS MODES
# data-mysql-0      Bound    pvc-abc123...      20Gi       RWO
```

Step 10: Update ConfigMap (Rolling Update)

bash

```
# Update ConfigMap
kubectl set env configmap/app-config LOG_LEVEL=DEBUG

# Or edit directly
kubectl edit configmap app-config

# Note: Pods don't automatically restart on ConfigMap change
# To force pod restart and reload config:
kubectl rollout restart deployment/config-demo-app

# Verify new LOG_LEVEL
kubectl exec $(kubectl get pods -l app=config-demo -o jsonpath='{.items[0].meta
```

Step 11: Clean Up

```
bash

# Delete all resources
kubectl delete deployment config-demo-app
kubectl delete pod data-pod
kubectl delete statefulset mysql
kubectl delete service mysql
kubectl delete pvc --all
kubectl delete configmap app-config
kubectl delete secret db-credentials db-credentials-secure mysql-secret
```

Lab 3 Summary:

- ✓ Created ConfigMap for application configuration
- ✓ Created Secret for sensitive data
- ✓ Mounted ConfigMap and Secret in pods
- ✓ Created PersistentVolumeClaim for storage
- ✓ Deployed stateful application with persistent storage
- ✓ Verified data persistence across pod restarts
- ✓ Created StatefulSet with dynamic storage provisioning

LAB 4: NETWORKING AND SERVICES

Lab Objective

Understand Kubernetes networking, service types, and network policies.

Step 1: Create Microservices Architecture

Create `microservices.yaml`:

yaml

```
# Frontend Service
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
spec:
  replicas: 2
  selector:
    matchLabels:
      app: frontend
  template:
    metadata:
      labels:
        app: frontend
    spec:
      containers:
        - name: frontend
          image: gcr.io/google-samples/hello-app:1.0
          ports:
            - containerPort: 8080
          env:
            - name: BACKEND_SERVICE
              value: "backend:8080"

---
apiVersion: v1
kind: Service
metadata:
  name: frontend
spec:
  type: LoadBalancer
  selector:
    app: frontend
  ports:
    - port: 80
      targetPort: 8080

# Backend Service
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
```

```
spec:
  replicas: 3
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
        - name: backend
          image: gcr.io/google-samples/hello-app:2.0
          ports:
            - containerPort: 8080

---
apiVersion: v1
kind: Service
metadata:
  name: backend
spec:
  type: ClusterIP
  selector:
    app: backend
  ports:
    - port: 8080
      targetPort: 8080

# Database Service
---
apiVersion: v1
kind: Pod
metadata:
  name: database
  labels:
    app: database
spec:
  containers:
    - name: database
      image: postgres:13
      ports:
        - containerPort: 5432
      env:
        - name: POSTGRES_PASSWORD
```

```
        value: "dbpassword"

---
apiVersion: v1
kind: Service
metadata:
  name: database
spec:
  type: ClusterIP
  selector:
    app: database
  ports:
    - port: 5432
      targetPort: 5432
```

Deploy microservices:

```
bash

# Create all services
kubectl apply -f microservices.yaml

# Wait for deployments
kubectl wait --for=condition=available --timeout=300s deployment/frontend
kubectl wait --for=condition=available --timeout=300s deployment/backend

# Verify all services
kubectl get services
kubectl get pods
```

Step 2: Test Service Discovery

```
bash
```

```
# Access frontend pod
FRONTEND_POD=$(kubectl get pods -l app=frontend -o jsonpath='{.items[0].metadata.name}')

# Test DNS resolution from frontend pod
kubectl exec $FRONTEND_POD -- nslookup backend

# Expected output:
# Name: backend.default.svc.cluster.local
# Address: 10.x.x.x (ClusterIP)

# Test connection to backend service
kubectl exec $FRONTEND_POD -- wget -O- http://backend:8080

# Test connection to database service
kubectl exec $FRONTEND_POD -- nc -zv database 5432
```

Step 3: Examine Service Details

```
bash

# Get service endpoints
kubectl get endpoints

# Expected output:
# NAME           ENDPOINTS                                     AGE
# backend        10.0.1.2:8080,10.0.1.3:8080...
# database       10.0.1.4:5432
# frontend       10.0.2.5:8080,10.0.2.6:8080

# Get frontend service with LoadBalancer IP
FRONTEND_IP=$(kubectl get service frontend -o jsonpath='{.status.loadBalancer.ingress[0].ip}')
echo "Frontend accessible at: http://$FRONTEND_IP"

# Test from external
curl http://$FRONTEND_IP
```

Step 4: Create Network Policy (Restrict Traffic)

Create `network-policy.yaml`:

```
yaml
```

```
# Allow traffic from frontend to backend only
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: backend-policy
spec:
  podSelector:
    matchLabels:
      app: backend
  policyTypes:
  - Ingress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: frontend
    ports:
    - protocol: TCP
      port: 8080

# Allow traffic from backend to database only
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: database-policy
spec:
  podSelector:
    matchLabels:
      app: database
  policyTypes:
  - Ingress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: backend
    ports:
    - protocol: TCP
      port: 5432
```

Apply network policies:

```
bash
```

```
# Enable network policies on cluster (may need cluster update)
```

```
gcloud container clusters update $CLUSTER_NAME \  
  --region $REGION \  
  --enable-network-policy
```

```
# Apply policies
```

```
kubectl apply -f network-policy.yaml
```

```
# Verify policies
```

```
kubectl get networkpolicies
```

Step 5: Test Network Policy

```
bash
```

```
# Create a test pod in different namespace
```

```
kubectl create namespace test-namespace
```

```
kubectl run test-pod \  
  --image=busybox:1.28 \  
  --namespace=test-namespace \  
  --restart=Never \  
  -it -- /bin/sh
```

```
# From test pod, try to access backend
```

```
# wget -O- http://backend.default.svc.cluster.local:8080
```

```
# Expected: Connection timeout (blocked by network policy)
```

```
# Exit test pod
```

```
exit
```

Step 6: Ingress Controller (Advanced Routing)

Create `ingress.yaml`:

```
yaml
```

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress
  annotations:
    kubernetes.io/ingress.class: gce
    ingress.gcp.kubernetes.io/pre-shared-cert: "ssl-cert" # Optional SSL
spec:
  rules:
    - host: api.example.com
      http:
        paths:
          - path: /api/*
            pathType: ImplementationSpecific
            backend:
              service:
                name: backend
                port:
                  number: 8080
    - host: app.example.com
      http:
        paths:
          - path: /*
            pathType: ImplementationSpecific
            backend:
              service:
                name: frontend
                port:
                  number: 80
```

Deploy Ingress (requires external DNS setup):

```
bash
```

```
# Create ingress
kubectl apply -f ingress.yaml

# Get ingress details
kubectl get ingress

# Get ingress IP
INGRESS_IP=$(kubectl get ingress app-ingress -o jsonpath='{.status.loadBalancer
echo "Ingress IP: $INGRESS_IP"

# In production, point api.example.com and app.example.com to $INGRESS_IP
```

Step 7: Service Discovery Methods

DNS Service Discovery:

```
bash

# Format: service.namespace.svc.cluster.local

# Access from pod in same namespace
http://backend:8080
http://backend.default.svc.cluster.local:8080

# Access from pod in different namespace
http://backend.default.svc.cluster.local:8080
```

Environment Variables:

```
bash

# Kubernetes automatically injects environment variables
kubectl exec $FRONTEND_POD -- env | grep BACKEND

# Expected output:
# BACKEND_SERVICE_HOST=10.x.x.x
# BACKEND_SERVICE_PORT=8080
```

Step 8: Test Load Balancing

```
bash
```

```
# Make multiple requests to backend service
for i in {1..10}; do
    kubectl exec $FRONTEND_POD -- wget -O- http://backend:8080
done

# Requests are distributed across multiple backend pods
```

Step 9: Clean Up

```
bash

# Delete all networking resources
kubectl delete ingress app-ingress
kubectl delete networkpolicies --all
kubectl delete deployment frontend backend
kubectl delete pod database
kubectl delete service frontend backend database
kubectl delete namespace test-namespace
```

Lab 4 Summary:

- ✓ Deployed microservices architecture
- ✓ Tested Kubernetes service discovery
- ✓ Understood ClusterIP, LoadBalancer, and Ingress
- ✓ Implemented Network Policies for traffic restriction
- ✓ Verified load balancing across pods
- ✓ Learned DNS and environment variable service discovery

LAB 5: DEPLOYMENT STRATEGIES

Lab Objective

Implement and compare different deployment strategies for zero-downtime updates.

Step 1: Create Initial Deployment (v1)

Create `rolling-deployment.yaml`:

```
yaml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
  labels:
    app: web-app
spec:
  replicas: 4
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1          # Max 1 extra pod during update
      maxUnavailable: 0   # Keep all pods available
  selector:
    matchLabels:
      app: web-app
  template:
    metadata:
      labels:
        app: web-app
        version: v1
    spec:
      containers:
        - name: app
          image: gcr.io/google-samples/hello-app:1.0
          ports:
            - containerPort: 8080
          lifecycle:
            preStop:
              exec:
                command: ["/bin/sh", "-c", "sleep 15"] # Graceful shutdown
      readinessProbe:
        httpGet:
          path: /
          port: 8080
        initialDelaySeconds: 5
        periodSeconds: 5
      livenessProbe:
        httpGet:
          path: /
          port: 8080
        initialDelaySeconds: 15
        periodSeconds: 10
```

Deploy v1:

```
bash

# Create deployment
kubectl apply -f rolling-deployment.yaml

# Wait for deployment to be ready
kubectl wait --for=condition=available --timeout=300s deployment/web-app

# Verify 4 pods are running
kubectl get pods -l app=web-app

# Expose service
kubectl expose deployment web-app \
  --type=LoadBalancer \
  --port=80 \
  --target-port=8080 \
  --name=web-app-service

# Get external IP
EXTERNAL_IP=$(kubectl get service web-app-service -o jsonpath='{.status.loadBalancingIngressIPs[0].ip}')
echo "Access at: http://$EXTERNAL_IP"
```

Step 2: Monitor Rolling Update

Create a script to monitor the update in real-time.

Terminal 1: Monitor deployments

```
bash

watch kubectl get deployment web-app
```

Terminal 2: Monitor pods

```
bash

watch 'kubectl get pods -l app=web-app -o wide'
```

Terminal 3: Monitor events

```
bash
```

```
kubectl get events --watch
```

Terminal 4: Monitor traffic (continue making requests)

```
bash

# Make continuous requests from external
EXTERNAL_IP=$(kubectl get service web-app-service -o jsonpath='{.status.loadBalancerIP}')

while true; do
    curl -s http://$EXTERNAL_IP | grep Version
    sleep 1
done

# Should always get successful responses (no errors)
```

Step 3: Perform Rolling Update

Update the deployment to v2:

```
bash

# Method 1: Update image via kubectl set
kubectl set image deployment/web-app \
    app=gcr.io/google-samples/hello-app:2.0 \
    --record

# Watch the rolling update
# Pod count will temporarily increase by 1 (maxSurge=1)
# Old pods terminate as new pods become ready
# Process continues until all pods updated

# Expected progression:
# Initial: 4 pods (v1)
# During: 4-5 pods (mix of v1 and v2)
# End: 4 pods (v2)

# Verify update completed
kubectl rollout status deployment/web-app
```

Alternative: Update via manifest

```
bash
```

```
# Edit deployment manifest
kubectl set image deployment/web-app app=gcr.io/google-samples/hello-app:2.0

# Or manually edit
kubectl edit deployment web-app
# Change image line to: gcr.io/google-samples/hello-app:2.0
# Save and exit

# Apply new manifest
kubectl apply -f rolling-deployment-v2.yaml
```

Step 4: Verify Zero-Downtime Update

Check traffic during update:

```
bash

# From Terminal 4, requests should show mix of versions:
# Version: 1.0   (old pod)
# Version: 2.0   (new pod)
# Version: 2.0   (new pod)

# No connection errors or timeouts
# No 503 Service Unavailable errors
```

Step 5: Rollback Update

If there's an issue with v2, rollback to v1:

```
bash
```

```
# View rollout history
kubectl rollout history deployment/web-app

# Expected output:
# REVISION  CHANGE-CAUSE
# 1         <none>
# 2         kubectl set image deployment/web-app app=gcr.io/google-samples/hell

# Rollback to previous version
kubectl rollout undo deployment/web-app

# Verify rollback
kubectl rollout status deployment/web-app

# Check pods are back to v1
kubectl get pods -l app=web-app
```

Step 6: Blue-Green Deployment Strategy

Create two independent deployments (blue and green):

Create `blue-green.yaml`:

yaml

```
# Blue Deployment (current version)
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app-blue
spec:
  replicas: 4
  selector:
    matchLabels:
      app: web-app
      version: blue
  template:
    metadata:
      labels:
        app: web-app
        version: blue
    spec:
      containers:
        - name: app
          image: gcr.io/google-samples/hello-app:1.0
          ports:
            - containerPort: 8080

# Green Deployment (new version - not receiving traffic yet)
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app-green
spec:
  replicas: 4
  selector:
    matchLabels:
      app: web-app
      version: green
  template:
    metadata:
      labels:
        app: web-app
        version: green
    spec:
      containers:
        - name: app
```

```
    image: gcr.io/google-samples/hello-app:2.0
    ports:
      - containerPort: 8080

# Service pointing to blue (can switch to green)
---
apiVersion: v1
kind: Service
metadata:
  name: web-app-bg
spec:
  type: LoadBalancer
  selector:
    app: web-app
    version: blue # Currently routes to blue
  ports:
    - port: 80
      targetPort: 8080
```

Deploy blue-green:

```
bash

# Create both deployments
kubectl apply -f blue-green.yaml

# Wait for both deployments
kubectl wait --for=condition=available --timeout=300s deployment/web-app-blue
kubectl wait --for=condition=available --timeout=300s deployment/web-app-green

# Both deployments running, but traffic only to blue
kubectl get pods

# Test traffic only goes to blue
EXTERNAL_IP=$(kubectl get service web-app-bg -o jsonpath='{.status.loadBalancer')
for i in {1..5}; do
  curl -s http://$EXTERNAL_IP | grep Version
done

# All responses should be Version: 1.0
```

Step 7: Switch Traffic from Blue to Green

```
bash
```

```
# Update service to route to green
kubectl patch service web-app-bg -p '{"spec":{"selector":{"version":"green"}}}'

# Verify traffic switched
EXTERNAL_IP=$(kubectl get service web-app-bg -o jsonpath='{.status.loadBalancer
for i in {1..5}; do
  curl -s http://$EXTERNAL_IP | grep Version
done

# All responses should now be Version: 2.0

# If there's an issue, switch back
kubectl patch service web-app-bg -p '{"spec":{"selector":{"version":"blue"}}}'
```

Step 8: Canary Deployment Strategy

Create traffic split between canary and stable versions:

Create `canary.yaml`:

yaml

```
# Stable Deployment (90% traffic)
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app-stable
spec:
  replicas: 9
  selector:
    matchLabels:
      app: web-app
      track: stable
  template:
    metadata:
      labels:
        app: web-app
        track: stable
    spec:
      containers:
        - name: app
          image: gcr.io/google-samples/hello-app:1.0
          ports:
            - containerPort: 8080

# Canary Deployment (10% traffic)
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app-canary
spec:
  replicas: 1
  selector:
    matchLabels:
      app: web-app
      track: canary
  template:
    metadata:
      labels:
        app: web-app
        track: canary
    spec:
      containers:
        - name: app
```

```
    image: gcr.io/google-samples/hello-app:2.0
    ports:
      - containerPort: 8080

# Service routes to both (9 stable + 1 canary)
---
apiVersion: v1
kind: Service
metadata:
  name: web-app-canary
spec:
  type: LoadBalancer
  selector:
    app: web-app
  ports:
    - port: 80
      targetPort: 8080
```

Deploy canary:

```
bash

# Create deployments
kubectl apply -f canary.yaml

# Wait for deployments
kubectl wait --for=condition=available --timeout=300s deployment/web-app-stable
kubectl wait --for=condition=available --timeout=300s deployment/web-app-canary

# Check distribution (should be 9 stable + 1 canary)
kubectl get pods

# Test traffic distribution
EXTERNAL_IP=$(kubectl get service web-app-canary -o jsonpath='{.status.loadBalancerIngress}')
for i in {1..30}; do
  curl -s http://$EXTERNAL_IP | grep Version
done

# Expected output: ~27 Version 1.0 responses, ~3 Version 2.0 responses
# Shows 90% stable, 10% canary traffic split
```

Step 9: Clean Up

```
bash
```

```
# Delete deployments and services
kubectl delete deployment web-app web-app-blue web-app-green web-app-stable web
kubectl delete service web-app-service web-app-bg web-app-canary
```

Lab 5 Summary:

- ✓ Implemented Rolling Update (gradual pod replacement)
- ✓ Monitored zero-downtime deployments
- ✓ Performed rollbacks to previous versions
- ✓ Implemented Blue-Green deployment (instant cutover)
- ✓ Implemented Canary deployment (gradual traffic shift)
- ✓ Compared deployment strategies

LAB 6: MONITORING AND LOGGING

Lab Objective

Set up monitoring and logging for Kubernetes applications.

Step 1: Verify Monitoring is Enabled

```
bash

# Check if Cloud Monitoring is enabled
gcloud container clusters describe $CLUSTER_NAME \
  --region $REGION \
  --format="value(monitoringConfig.componentConfig.enableComponents)"

# Should include "SYSTEM_COMPONENTS", "WORKLOADS"

# If not enabled, enable it:
gcloud container clusters update $CLUSTER_NAME \
  --region $REGION \
  --enable-cloud-logging \
  --logging=SYSTEM_COMPONENTS,WORKLOADS \
  --enable-cloud-monitoring
```

Step 2: Deploy Application with Monitoring

Create `monitoring-app.yaml`:

```
yaml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: metrics-demo
spec:
  replicas: 2
  selector:
    matchLabels:
      app: metrics-demo
  template:
    metadata:
      labels:
        app: metrics-demo
    spec:
      containers:
        - name: app
          image: gcr.io/google-samples/hello-app:2.0
          ports:
            - containerPort: 8080
          resources:
            requests:
              cpu: 100m
              memory: 128Mi
            limits:
              cpu: 500m
              memory: 512Mi
          livenessProbe:
            httpGet:
              path: /
              port: 8080
            initialDelaySeconds: 15
            periodSeconds: 20
          readinessProbe:
            httpGet:
              path: /
              port: 8080
            initialDelaySeconds: 5
            periodSeconds: 10
```

Deploy:

```
bash
```

```
# Create deployment
kubectl apply -f monitoring-app.yaml

# Expose service
kubectl expose deployment metrics-demo \
  --type=LoadBalancer \
  --port=80 \
  --target-port=8080 \
  --name=metrics-service

# Generate traffic
EXTERNAL_IP=$(kubectl get service metrics-service -o jsonpath='{.status.loadBalancingIngressIPs[0].ip}')

# Continuous load generation (runs in background)
while true; do curl -s http://$EXTERNAL_IP > /dev/null; sleep 1; done &
```

Step 3: View Metrics in Cloud Console

```
bash

# Navigate to Cloud Monitoring in console
# https://console.cloud.google.com/monitoring

# Or use gcloud to view metrics
gcloud monitoring metrics-descriptors list --filter="resource.type=k8s_container"

# Query specific metric
gcloud monitoring read \
  kubernetes.io/container/memory/used_bytes \
  --filter='resource.labels.pod_name="metrics-demo-*"'
```

Step 4: View Logs in Cloud Console

```
bash
```

```
# Navigate to Cloud Logging in console
# https://console.cloud.google.com/logs

# Filter logs for specific pod
gcloud logging read 'resource.type="k8s_pod" AND resource.labels.pod_name="metr
  --limit=10 \
  --format=json

# View pod logs using kubectl
kubectl logs -l app=metrics-demo --tail=50

# Stream logs in real-time
kubectl logs -f -l app=metrics-demo
```

Step 5: Create Custom Monitoring Dashboard

```
bash
```

```

# Create dashboard JSON
cat > dashboard.json << 'EOF'
{
  "displayName": "Kubernetes Cluster Dashboard",
  "dashboardFilters": [],
  "gridLayout": {
    "widgets": [
      {
        "title": "CPU Usage",
        "xyChart": {
          "dataSets": [
            {
              "timeSeriesQuery": {
                "timeSeriesFilter": {
                  "filter": "metric.type=\"kubernetes.io/node/cpu/allocatable_u
                }
              }
            }
          ]
        }
      },
      {
        "title": "Memory Usage",
        "xyChart": {
          "dataSets": [
            {
              "timeSeriesQuery": {
                "timeSeriesFilter": {
                  "filter": "metric.type=\"kubernetes.io/node/memory/allocatabl
                }
              }
            }
          ]
        }
      },
      {
        "title": "Pod Count",
        "xyChart": {
          "dataSets": [
            {
              "timeSeriesQuery": {
                "timeSeriesFilter": {
                  "filter": "metric.type=\"kubernetes.io/container/uptime\" res
                }
              }
            }
          ]
        }
      }
    ]
  }
}

```

```
    }
  }
]
}
}
]
}
}
EOF

# Create dashboard
gcloud monitoring dashboards create --config-from-file=dashboard.json
```

Step 6: Set Up Alerts

```
bash

# Create alert policy for high CPU
gcloud alpha monitoring policies create \
  --notification-channels=CHANNEL_ID \
  --display-name="High CPU Usage" \
  --condition-display-name="CPU > 80%" \
  --condition-threshold-value=0.8 \
  --condition-threshold-duration=300s \
  --condition-threshold-filter='metric.type="kubernetes.io/container/cpu/core'
```

Step 7: View Cluster Events

```
bash

# Get all cluster events
kubectl get events

# Watch events in real-time
kubectl get events --watch

# Get events for specific resource
kubectl describe pod metrics-demo-xxxx

# Search for warning events
kubectl get events --field-selector type=Warning
```

Step 8: Enable Pod Logging

```
bash

# View logs from pod directly
kubectl logs metrics-demo-xxxx

# Tail logs
kubectl logs -f metrics-demo-xxxx

# View logs from previous container (if crashed)
kubectl logs metrics-demo-xxxx --previous

# View logs from all pods with label
kubectl logs -l app=metrics-demo

# View logs with timestamp
kubectl logs -f metrics-demo-xxxx --timestamps=true
```

Step 9: Debug Using Metrics

```
bash

# Check resource usage
kubectl top nodes
kubectl top pods

# Get detailed pod metrics
kubectl describe pod metrics-demo-xxxx

# Check events
kubectl get events --sort-by='.lastTimestamp'

# Check node conditions
kubectl get nodes -o wide
```

Step 10: Create Custom Metrics (Advanced)

```
bash
```

```
# View all available metrics
kubectl get --raw /apis/custom.metrics.k8s.io/v1beta1

# Query metrics from application
# Your app needs to expose Prometheus metrics on /metrics endpoint

# Example: If app exposes custom metric "http_requests_total"
kubectl get --raw "/apis/custom.metrics.k8s.io/v1beta1/namespaces/default/pods/
```

Step 11: Export and Analyze Logs

```
bash

# Export logs to Cloud Storage
gcloud logging sinks create log-export-bucket \
  gs://my-bucket-name \
  --log-filter='resource.type="k8s_pod"'

# Export logs to BigQuery for analysis
gcloud logging sinks create log-export-bq \
  bigquery.googleapis.com/projects/$PROJECT_ID/datasets/k8s_logs \
  --log-filter='resource.type="k8s_pod"'

# Query exported logs in BigQuery
bq query --use_legacy_sql=false \
  'SELECT timestamp, jsonPayload.message FROM `project.dataset.k8s_logs` LIMIT
```

Step 12: Clean Up

```
bash

# Stop background load generation
pkill -f "while true"

# Delete deployment and service
kubectl delete deployment metrics-demo
kubectl delete service metrics-service

# Delete dashboard and alerts (optional)
gcloud monitoring dashboards delete DASHBOARD_ID
gcloud alpha monitoring policies delete POLICY_ID
```

Lab 6 Summary:

- ✓ Verified monitoring is enabled on cluster
- ✓ Deployed monitored application
- ✓ Viewed metrics in Cloud Console
- ✓ Viewed and filtered logs
- ✓ Created custom monitoring dashboard
- ✓ Set up alerts for resource thresholds
- ✓ Debugged using events and metrics
- ✓ Exported logs for analysis

PRODUCTION PATTERNS

1. High Availability Architecture

yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: production-app
spec:
  replicas: 3 # Minimum 3 for high availability
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0 # Always have 2 running pods
  selector:
    matchLabels:
      app: production-app
  template:
    metadata:
      labels:
        app: production-app
      annotations:
        prometheus.io/scrape: "true"
        prometheus.io/port: "8080"
    spec:
      # Pod anti-affinity spreads pods across nodes
      affinity:
        podAntiAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
            - labelSelector:
                matchExpressions:
                  - key: app
                    operator: In
                    values:
                      - production-app
              topologyKey: kubernetes.io/hostname

      # Graceful shutdown
      terminationGracePeriodSeconds: 30

    containers:
      - name: app
        image: gcr.io/my-project/production-app:v1.0
        imagePullPolicy: Always
        ports:
          - containerPort: 8080
            name: http
```

```
- containerPort: 9090
  name: metrics # Prometheus metrics

# Resource limits prevent node overload
resources:
  requests:
    cpu: 500m
    memory: 512Mi
  limits:
    cpu: 1000m
    memory: 1Gi

# Health checks
livenessProbe:
  httpGet:
    path: /healthz
    port: 8080
  initialDelaySeconds: 30
  periodSeconds: 10
  timeoutSeconds: 5
  failureThreshold: 3

readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  initialDelaySeconds: 10
  periodSeconds: 5
  timeoutSeconds: 2
  failureThreshold: 3

# Graceful shutdown hook
lifecycle:
  preStop:
    exec:
      command: ["/bin/sh", "-c", "sleep 15"]
```

2. Resource Management

yaml

```
# Namespace with resource quotas
---
apiVersion: v1
kind: Namespace
metadata:
  name: production

---
apiVersion: v1
kind: ResourceQuota
metadata:
  name: prod-quota
  namespace: production
spec:
  hard:
    requests.cpu: "10"           # Max 10 CPUs
    requests.memory: "20Gi"      # Max 20 GB memory
    limits.cpu: "20"
    limits.memory: "40Gi"
    pods: "100"
    services.loadbalancers: "5"

---
apiVersion: v1
kind: LimitRange
metadata:
  name: prod-limits
  namespace: production
spec:
  limits:
    - default:
        cpu: "1"
        memory: "512Mi"
      defaultRequest:
        cpu: "250m"
        memory: "256Mi"
      max:
        cpu: "2"
        memory: "1Gi"
      min:
        cpu: "50m"
        memory: "64Mi"
    type: Container
```

3. Network Security

yaml

```
# Deny all ingress by default
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: deny-all
  namespace: production
spec:
  podSelector: {}
  policyTypes:
  - Ingress

# Allow traffic from ingress controller
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-ingress
  namespace: production
spec:
  podSelector:
    matchLabels:
      app: web-app
  policyTypes:
  - Ingress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: ingress-nginx
    ports:
    - protocol: TCP
      port: 8080

# Allow traffic between services
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-internal
  namespace: production
spec:
  podSelector: {}
  policyTypes:
```

```
- Ingress
ingress:
- from:
  - namespaceSelector:
      matchLabels:
        name: production
```

4. Multi-Region Deployment

yaml

```

# Deployment in Region 1
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app-us-central
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web-app
  template:
    metadata:
      labels:
        app: web-app
        region: us-central
    spec:
      affinity:
        nodeAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
            nodeSelectorTerms:
              - matchExpressions:
                  - key: cloud.google.com/gke-nodepool
                    operator: In
                    values:
                      - us-central-pool
      # ... pod spec

# Traffic split across regions via Service
---
apiVersion: v1
kind: Service
metadata:
  name: app-global
spec:
  selector:
    app: web-app # Selects from both regions
  type: LoadBalancer
# Cloud Load Balancer distributes traffic

```

TROUBLESHOOTING GUIDE

Pod Not Starting

Symptoms:

- Pod stuck in `Pending`, `ContainerCreating`, or `CrashLoopBackOff` state

Diagnosis:

```
bash
```

```
# Check pod status
```

```
kubectl describe pod <pod-name>
```

```
# Look for Events section showing why pod can't start
```

```
# Common causes:
```

```
# 1. Insufficient resources
```

```
#   → Check node capacity: kubectl top nodes
```

```
#   → Reduce resource requests or add more nodes
```

```
# 2. Image pull errors
```

```
#   → Verify image exists in registry
```

```
#   → Check image pull secrets
```

```
#   kubectl get secrets
```

```
# 3. Volume mount issues
```

```
#   → Check PVC exists: kubectl get pvc
```

```
#   → Check PV status: kubectl get pv
```

```
#   → Verify access permissions
```

```
# 4. Node affinity mismatch
```

```
#   → Check node labels: kubectl get nodes --show-labels
```

```
#   → Verify pod affinity rules match available nodes
```

Service Not Accessible

Symptoms:

- Can't access service from external or internal clients
- Connection timeout or refused

Diagnosis:

```
bash
```

```
# Check service exists and has endpoints
kubectl get service <service-name>
kubectl get endpoints <service-name>

# If endpoints are empty:
# → Check pod selector matches deployed pods
# → Verify pods are in Running state
# → Check pod labels: kubectl get pods --show-labels

# Check service DNS resolution
kubectl run -it debug --image=busybox --restart=Never -- nslookup <service-name>

# Check network policies aren't blocking traffic
kubectl get networkpolicies

# If LoadBalancer type:
# → Check external IP is assigned (may take 1-2 min)
# → Verify firewall rules allow traffic
# → gcloud compute firewall-rules list
```

High Resource Usage

Symptoms:

- Nodes running out of CPU or memory
- Pods being evicted
- Cluster slow or unresponsive

Diagnosis:

```
bash
```

```
# Check resource usage
kubectl top nodes
kubectl top pods

# Identify which pods use most resources
kubectl top pods --all-namespaces | sort --reverse -k 3 | head -10

# Check pod resource requests vs actual usage
kubectl describe pod <pod-name>

# Solutions:
# 1. Increase pod resource limits
#   → Edit deployment, update resources.limits

# 2. Scale down pods
#   → kubectl scale deployment <name> --replicas=<number>

# 3. Add more nodes
#   → Check cluster autoscaler is enabled
#   → May take 2-3 minutes for new nodes

# 4. Optimize application
#   → Profile application for memory leaks
#   → Reduce log verbosity
```

Deployment Stuck or Won't Update

Symptoms:

- Deployment update takes too long or hangs
- Old pods never terminate
- New pods never become ready

Diagnosis:

```
bash
```

```
# Check deployment status
kubectl rollout status deployment/<name>

# Check recent events
kubectl describe deployment <name>

# If pods are stuck in `Terminating`:
# → Check preStop hooks: kubectl get pod -o yaml
# → May need to force delete: kubectl delete pod <name> --grace-period=0 --for

# If new pods won't become ready:
# → Check readiness probe: kubectl describe pod <pod-name>
# → Check application logs: kubectl logs <pod-name>

# Solutions:
# 1. Increase readiness probe timeout
# → Update deployment readinessProbe.initialDelaySeconds

# 2. Check application is serving requests
# → curl from pod to verify endpoint works

# 3. Adjust pod disruption budget if set
# → kubectl get poddisruptionbudgets
```

DNS Resolution Issues

Symptoms:

- Pods can't resolve service DNS names
- Connection errors like "nodename or servname not known"

Diagnosis:

```
bash
```

```
# Test DNS from pod
kubectl run -it debug --image=busybox --restart=Never -- /bin/sh

# From pod:
# nslookup service-name
# nslookup service-name.default.svc.cluster.local
# cat /etc/resolv.conf

# Check CoreDNS is running
kubectl get pods -n kube-system -l k8s-app=kube-dns

# Check CoreDNS logs
kubectl logs -n kube-system -l k8s-app=coredns

# Solutions:
# 1. Restart CoreDNS
#   → kubectl rollout restart deployment/coredns -n kube-system

# 2. Check network connectivity
#   → Verify pods can communicate: kubectl exec pod1 -- ping pod2

# 3. Rebuild pod resolv.conf
#   → Restart pod: kubectl delete pod <pod-name>
```

Persistent Storage Issues

Symptoms:

- Pod stuck in `Pending` with PVC
- `Pod unbound PVC` errors
- Data lost after pod restart

Diagnosis:

```
bash
```

```
# Check PVC status
kubectl get pvc
kubectl describe pvc <pvc-name>

# Check PV status
kubectl get pv
kubectl describe pv <pv-name>

# If PVC is `Pending`:
# → Check storage class exists: kubectl get storageclass
# → Check disk quota: gcloud compute disks list
# → Check region/zone match: kubectl get nodes -o wide

# If data is lost:
# → Verify accessMode is correct (ReadWriteOnce vs ReadWriteMany)
# → Check pod has volumeMounts section
# → Verify mount path is correct

# Solutions:
# 1. Recreate PVC
# → kubectl delete pvc <name>
# → kubectl apply -f pvc.yaml

# 2. Manual cleanup of orphaned PVs
# → gcloud compute disks delete <disk-name>
```

Cluster Connectivity Issues

Symptoms:

- kubectl commands timeout
- "Connection refused" errors
- Can't authenticate to cluster

Diagnosis:

```
bash
```

```
# Check cluster status
gcloud container clusters describe <cluster-name> --region <region>

# Verify credentials
gcloud container clusters get-credentials <cluster-name> --region <region>

# Check kubectl config
kubectl config view
kubectl config current-context

# Test connectivity
kubectl cluster-info
kubectl get nodes

# If master is down:
#   → Check cluster status on GCP Console
#   → Automatic repairs may be in progress
#   → Contact support if extends beyond 30 minutes

# Solutions:
# 1. Update credentials
#   → gcloud container clusters get-credentials <name> --region <region> --for

# 2. Check firewall rules
#   → gcloud compute firewall-rules list
#   → Allow port 443 from your IP

# 3. Verify API is enabled
#   → gcloud services enable container.googleapis.com
```

SUMMARY

What You've Learned

✓ Kubernetes Fundamentals

- Pods, Deployments, StatefulSets, Services
- ConfigMaps and Secrets for configuration
- Persistent storage and networking

✓ GKE Cluster Management

- Creating and configuring GKE clusters
- Node pools and cluster autoscaling

- Integrating with Google Cloud services

✓ **Scaling & Load Balancing**

- Horizontal Pod Autoscaler (HPA)
- Cluster autoscaling
- Multiple service types and load balancing

✓ **Deployment Strategies**

- Rolling updates with zero downtime
- Blue-green deployments for instant cutover
- Canary deployments for gradual rollout

✓ **Monitoring & Logging**

- Cloud Monitoring and Cloud Logging
- Creating dashboards and alerts
- Debugging with logs and metrics

✓ **Production Patterns**

- High availability with pod anti-affinity
- Resource management and quotas
- Network policies and security

Next Steps

1. **Deploy to Production:** Apply these concepts to production workloads
2. **Automate:** Use Terraform and Cloud Build for IaC and CI/CD
3. **Optimize:** Monitor and tune performance and costs
4. **Secure:** Implement additional security layers (Workload Identity, RBAC, Pod Security Policy)
5. **Scale:** Deploy across regions and manage global infrastructure

Additional Resources

- [Kubernetes Documentation](#)
- [Google Kubernetes Engine Documentation](#)
- [kubectl Cheat Sheet](#)
- [GKE Best Practices](#)

Congratulations on completing the Kubernetes Engine Tutorial! 🎉

You now have practical knowledge to deploy, manage, and scale containerized applications on Google Kubernetes Engine. Keep practicing and exploring advanced features!